

ReportMaster AI

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering
School of Engineering and Sciences



SRM UNIVERSITY AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

April, 2026

Submitted by

Name	Roll Number
Nikhil Kumar	AP23110010119
Rishi Kumar	AP23110011088
Rishik Gorakala	AP23110010446
Suhail Mohammad	AP23110011156
Sriya Reddy Tadiparthi	AP23110011159

Report ToneCraft AI: Emotion-Aware Response Composer

Project Description:

Project Overview

ToneCraft AI: Emotion-Aware Response Composer is an AI-powered web application designed to assist customer support agents in generating emotionally intelligent, context-aware, and professionally structured responses.

The system analyzes customer messages to detect sentiment and emotional state, and then generates appropriate replies tailored to different tones and communication channels.

In modern customer service environments, agents frequently handle emotionally charged messages such as complaints, urgent requests, or dissatisfaction. Crafting the right response requires understanding the customer's emotional context, maintaining brand tone, and ensuring clarity and professionalism. ToneCraft AI addresses this challenge by automating the process of response generation while preserving empathy and consistency.

The system is built using a **React-based frontend** for user interaction and a **FastAPI backend** for processing and API management. It integrates **Natural Language Processing (NLP)** techniques and **Large Language Models (LLMs)** to analyze text and generate human-like responses. Additionally, the system includes a **rule-based fallback mechanism**, ensuring continuous operation even when external AI APIs are unavailable.

ToneCraft AI provides several key capabilities, including:

- Detection of sentiment (Positive, Neutral, Negative)
- Identification of emotions such as anger, frustration, and urgency
- Generation of multiple response types (main, short, detailed, de-escalation)
- Brand tone customization (Professional, Warm, Apologetic, etc.)
- Support for multiple communication channels (Email, Chat, WhatsApp, Social Media)
- Case saving and retrieval for future reference

The application is deployed using **Vercel**, making it accessible as a web-based platform without requiring installation.

Its lightweight design ensures that it can run efficiently on standard hardware while delivering real-time results.

Overall, ToneCraft AI serves as a practical solution for improving communication quality in customer support systems. By combining emotional intelligence with AI-driven automation, it helps reduce response time, maintain consistency, and enhance customer satisfaction.

1. Problem Statement

In today's digital world, effective communication plays a critical role in customer satisfaction and business success. Customer support agents and professionals frequently interact with users through emails, chat systems, and support tickets. However, maintaining the correct tone and emotional balance in responses remains a significant challenge.

Many users struggle to draft responses that are **clear, professional, and emotionally appropriate**, especially when dealing with frustrated or dissatisfied customers. In such situations, responses may unintentionally sound rude, insensitive, or overly formal, which can negatively impact the customer experience.

Some of the major issues observed in existing communication systems include:

- **Lack of Emotional Intelligence:** Traditional systems do not understand the emotional context of user input, leading to generic or inappropriate replies.
- **Inconsistent Tone:** Responses may vary in tone depending on the individual, resulting in inconsistent communication across the organization.
- **Time-Consuming Response Drafting:** Writing well-structured and empathetic replies requires time and effort, especially under pressure.
- **Ineffective Handling of Negative Sentiment:** Agents often find it difficult to respond appropriately to angry or frustrated customers.
- **Limited Automation:** Existing tools focus mainly on grammar correction rather than tone and emotion adjustment.

Due to these limitations, organizations face challenges such as reduced customer satisfaction, increased complaint escalation, and poor communication quality.

To address these challenges, there is a need for an intelligent system that can:

- Analyze the emotional tone and intent of user input
- Generate context-aware and empathetic responses
- Provide multiple tone variations for flexibility
- Assist users in maintaining consistent and professional communication

ToneCraft AI: Emotion-Aware Response Composer is proposed as a solution to these problems. The system leverages advanced Artificial Intelligence techniques, including Natural Language Processing (NLP), sentiment analysis, and large language models, to automatically transform user messages into emotionally intelligent and professionally appropriate responses.

This project aims to bridge the gap between human emotional understanding and automated communication systems, thereby improving overall communication quality and customer experience.

2. Objectives

The primary objective of *ToneCraft AI: Emotion-Aware Response Composer* is to develop an

intelligent system that enhances the quality of written communication by generating responses that are context-aware, emotionally appropriate, and professionally structured. The system is designed to assist users, especially customer support agents, in crafting effective replies efficiently.

The key objectives of the project are as follows:

1. To Analyze User Input and Detect Sentiment

The system aims to analyze the input text using Natural Language Processing (NLP) techniques to identify the underlying sentiment and emotional tone, such as positive, negative, or neutral, along with emotions like anger, frustration, or satisfaction.

2. To Generate Emotionally Appropriate Responses

Based on the detected sentiment and context, the system generates responses that are empathetic, polite, and aligned with the emotional state of the user, ensuring better communication.

3. To Provide Multiple Tone Variations

The project aims to offer different tone options such as formal, casual, friendly, polite, and persuasive, allowing users to choose the most suitable response based on the situation.

4. To Improve Communication Quality

The system focuses on enhancing grammar, clarity, and structure of the text while maintaining the original meaning, thereby improving overall communication effectiveness.

5. To Reduce Response Time

By automating the response generation process, the system helps users quickly generate high-quality replies, reducing the time required for drafting messages manually.

6. To Ensure Consistency in Tone

The project aims to maintain a consistent tone across all communications, which is particularly important in professional and organizational environments.

7. To Develop a User-Friendly Web Application

The system is designed as a web-based application with an intuitive interface, making it easy for users to input text, select tone preferences, and obtain results instantly.

8. To Leverage Advanced AI Technologies

The project aims to utilize modern AI technologies such as sentiment analysis models, NLP techniques, and Large Language Model (LLM) APIs to build an efficient and intelligent system.

9. To Enhance Customer Experience

By enabling better communication, the system ultimately aims to improve customer satisfaction, reduce misunderstandings, and build stronger user relationships.

10. To Create a Scalable and Extendable System

The project is designed in a way that it can be extended in the future with additional features such as multilingual support, voice-based interaction, and deeper emotion detection.

3. Core Technologies Used

<u>Technology</u>	<u>Purpose</u>
Python	Handles backend logic, data processing, and integration of AI components such as NLP models and APIs.
FastAPI	Provides a high-performance, asynchronous API framework for efficient communication between frontend and backend.
Sentiment Analysis Models	Analyzes user input to detect sentiment (positive, negative, neutral) and emotional tone for appropriate response generation.
Natural Language Processing (NLP)	Processes and understands human language, including text preprocessing, intent recognition, and context understanding.
Large Language Model (LLM) APIs (e.g., GPT / Gemini)	Generates human-like, context-aware, and tone-adjusted responses based on user input.

4. Key Features

The *ToneCraft AI: Emotion-Aware Response Composer* system offers a range of intelligent and user-centric features designed to improve communication quality and efficiency. The key features of the system are as follows:

1. Tone Transformation

The system allows users to convert their input text into multiple tones such as **formal, casual, friendly, polite, and persuasive**. This helps users adapt their communication style based on the context and audience.

2. Emotion-Aware Response Generation

The system analyzes the emotional context of the input message and generates responses that are empathetic and appropriate. This ensures that replies align with the user's sentiment, especially in sensitive or negative situations.

3. AI-Based Text Refinement

ToneCraft AI improves the overall quality of the text by correcting grammar, enhancing clarity, and restructuring sentences while preserving the original meaning of the message.

4. Multiple Response Suggestions

The system provides multiple alternative responses for a single input, giving users the flexibility to choose the most suitable reply according to their needs.

5. Real-Time Processing

The application generates results instantly after receiving input, enabling quick and efficient response creation without delays.

6. User-Friendly Interface

The web-based interface is designed to be simple and intuitive, allowing users to easily input text, select tone preferences, and obtain results without technical knowledge.

7. Context-Aware Understanding

The system uses advanced Natural Language Processing (NLP) techniques to understand the intent and context of the input text, ensuring that generated responses are relevant and meaningful.

8. Consistency in Communication

By standardizing tone and structure, the system helps maintain consistent communication across different users and

scenarios, which is essential in professional environments.

9. Scalable and Extendable Design

The system is designed to be scalable and can be extended with additional features such as multilingual support, voice-based input, and deeper emotion detection in future developments.

10. Web-Based Accessibility

Being a web application, ToneCraft AI can be accessed from anywhere without the need for installation, making it convenient and widely usable

Application Scenarios / Use Cases

The *ToneCraft AI: Emotion-Aware Response Composer* can be applied across various domains where effective and emotionally intelligent communication is essential. The system is particularly useful in environments that involve frequent interaction with customers or users. The major application scenarios are as follows:

1. Customer Support Centers

ToneCraft AI is highly beneficial for customer support teams handling large volumes of queries and complaints. It helps agents:

- Respond to frustrated or angry customers with empathy
- Maintain a consistent tone across all communications
- Reduce response time while improving quality

This leads to better customer satisfaction and efficient issue resolution.

2. E-commerce Platforms

Online shopping platforms frequently deal with customer issues such as delayed deliveries, refunds, and product complaints. ToneCraft AI helps:

- Generate polite and reassuring responses
- Handle negative reviews and complaints professionally
- Improve brand reputation through better communication

3. Banking and Financial Services

In banking and financial services, communication must be clear, professional, and sensitive. ToneCraft AI assists in:

- Responding to customer concerns regarding transactions, delays, or account issues
- Maintaining a formal and secure communication tone
- Avoiding miscommunication in critical financial matters

4. SaaS and IT Support Services

Software companies and IT support teams can use ToneCraft AI to:

- Respond to technical queries in a user-friendly manner
- Simplify complex explanations for customers
- Handle urgent and critical support tickets effectively

5. Social Media Management

Brands often receive feedback, complaints, and queries on social media platforms. ToneCraft AI helps:

- Generate quick and appropriate replies
- Manage public communication professionally
- Handle negative comments without escalating conflicts

6. Email Communication in Organizations

Professionals often need to send emails that require the right tone. ToneCraft AI can:

- Convert informal messages into formal emails
- Add politeness and clarity to communication
- Ensure professionalism in business correspondence

7. Helpdesk and Ticketing Systems

The system can be integrated into helpdesk platforms to:

- Automatically suggest responses for support tickets
- Improve consistency in replies across agents
- Assist new agents with guided communication

8. Training and Skill Development for Support Agents

ToneCraft AI can also be used as a training tool:

- Helps new agents learn how to respond appropriately
- Provides “Do and Don’t” guidance
- Improves emotional intelligence and communication skills

9. Complaint Management Systems

Organizations handling complaints can use the system to:

- De-escalate sensitive situations
- Respond to dissatisfied customers effectively
- Maintain a calm and professional tone in all communications

10. Multi-Channel Customer Communication

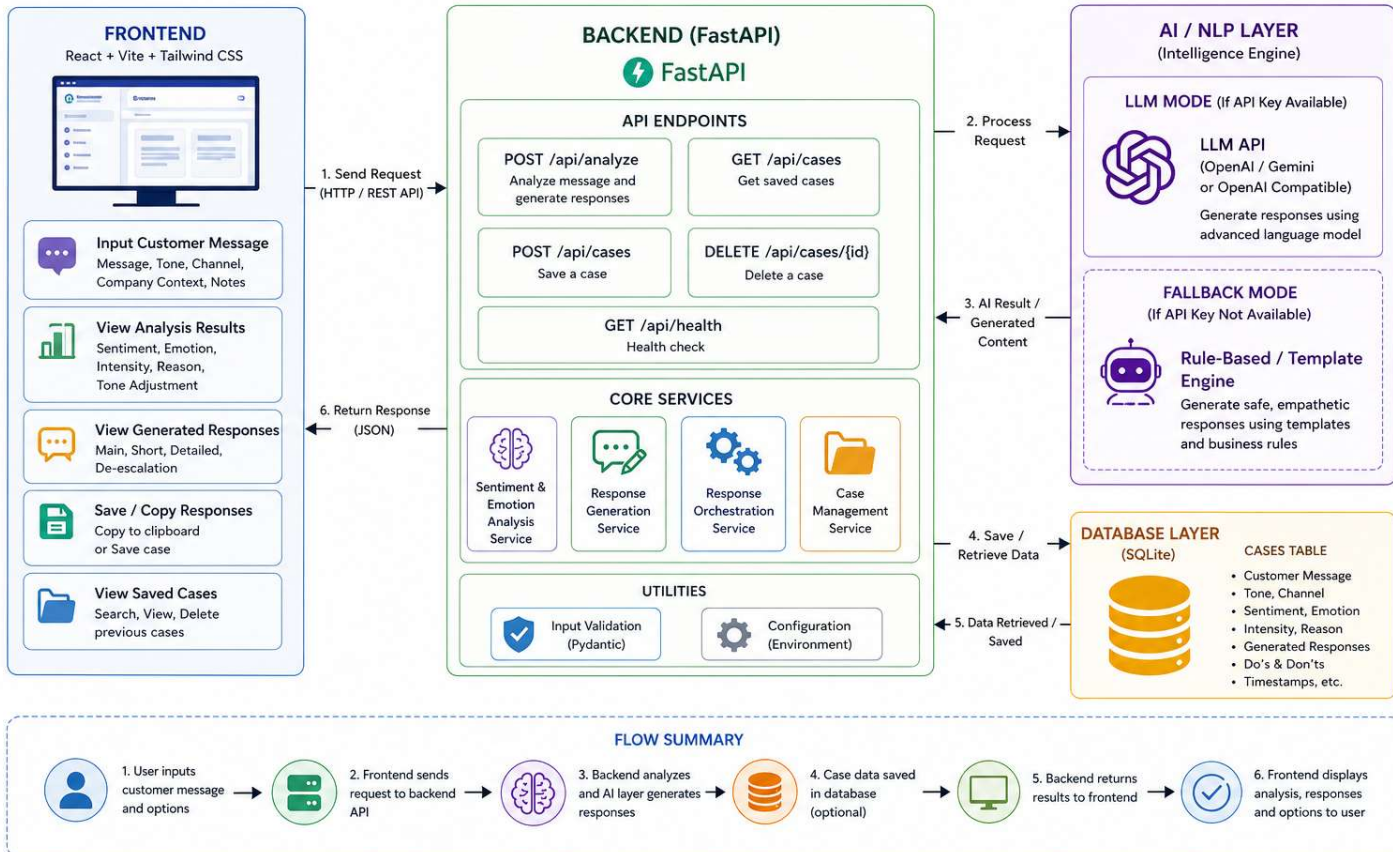
ToneCraft AI supports multiple communication channels such as:

- Email
- Chat
- WhatsApp
- Social Media

This makes it suitable for businesses that operate across different platforms and require consistent communication.

TECHNICAL ARCHITECTURE :

TONECRAFT AI - TECHNICAL ARCHITECTURE



TECHNICAL ARCHITECTURE :

The working flow of *ToneCraft AI: Emotion-Aware Response Composer* follows a structured pipeline from user input to response generation and display. The system ensures efficient communication between frontend, backend, AI processing layer, and database.

Step 1: User Input (Frontend Layer)

The process begins when the support agent enters a customer message into the web application. Along with the message, the user may also select:

- Brand tone (e.g., Professional, Friendly, Apologetic)
- Communication channel (Email, Chat, WhatsApp, etc.)
- Optional company context
- Optional agent notes

The frontend interface (built using React) collects this data and prepares it for processing.

Step 2: Request Sent to Backend (API Call)

The frontend sends the collected data to the backend server using a REST API request:

- Endpoint used: **POST /api/analyze**
- Data format: JSON

This request is transmitted over HTTP to the FastAPI backend.

Step 3: Backend Validation and Routing

Upon receiving the request, the backend performs:

- Input validation using Pydantic schemas
- Verification of required fields
- Routing the request to the appropriate service modules

This ensures that only valid and structured data is processed further.

Step 4: Sentiment and Emotion Analysis

The backend invokes the **Sentiment & Emotion Analysis Service**, which:

- Determines sentiment (Positive / Neutral / Negative)
- Detects emotion (Anger, Frustration, Urgency, etc.)
- Calculates emotional intensity (0–100)
- Generates a reason for the detected emotion

This step provides the emotional context required for response generation.

Step 5: Response Generation (AI Layer)

The system generates responses using one of the following modes:

LLM Mode (if API key is available):

- Sends prompt to LLM (e.g., OpenAI / Gemini)
- Generates intelligent, context-aware responses

Fallback Mode (if API key is not available):

- Uses rule-based templates
- Generates safe and empathetic responses

The system produces:

- Main reply
- Short reply
- Detailed reply
- De-escalation reply

Step 6: Response Orchestration

The backend combines all outputs into a structured format, including:

- Sentiment and emotion results
- Intensity score and explanation
- Tone adjustment recommendations
- Generated responses
- Do and Don't guidelines

Step 7: Database Interaction (Optional)

If the user chooses to save the case:

- The backend stores data in **SQLite database**
- Includes message, analysis results, and generated responses
- Enables retrieval of past cases

Step 8: Response Sent to Frontend

The backend sends the final processed result back to the frontend in JSON format.

Step 9: Display of Results (Frontend Layer)

The frontend displays:

- Sentiment and emotion analysis
- Emotional intensity (with progress bar)
- Generated responses
- Copy and save options

Users can:

- Copy responses
- Save cases
- View previous cases

Pre-requisites:

The successful development and deployment of *ToneCraft AI: Emotion-Aware Response Composer* require specific software tools, technical knowledge, system configurations, and dependencies. These prerequisites are based on the actual architecture and implementation of the project.

1. Software Requirements

The following software components are required to build and run the system:

- **Python (3.8 or above)**
Used for backend development, API handling, sentiment analysis, and response generation logic.
- **Node.js (v16 or above)**
Required for running the frontend application built using React and Vite.
- **Code Editor / IDE**
Recommended tools include Visual Studio Code or PyCharm for efficient development.
- **Web Browser**
Modern browsers such as Google Chrome, Microsoft Edge, or Firefox for accessing and testing the application.
- **Git (Optional)**
Used for version control and project management.

2. Technical Knowledge Requirements

To effectively understand and implement this project, the following knowledge is required:

- **Programming Languages**
Basic knowledge of Python (backend) and JavaScript (frontend).
- **Web Development**
Understanding of HTML, CSS, React, and Vite for building user interfaces.
- **Backend Development**
Knowledge of FastAPI, REST APIs, and request-response handling.
- **Artificial Intelligence and NLP**
Basic concepts of:
 - Sentiment Analysis
 - Emotion Detection
 - Generative AI
 - Prompt-based response generation
- **Database Management**
Basic understanding of SQLite and SQLAlchemy for storing and retrieving data.

3. Hardware Requirements

The system can run on standard computing devices with the following minimum specifications:

- **Processor:** Dual-core or higher
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** At least 1–2 GB free space
- **Internet Connection:** Required for API-based AI models and deployment

4. Project-Specific Dependencies

Backend Dependencies

- FastAPI
- Uvicorn
- Pydantic
- SQLAlchemy
- pytest

AI and NLP Components

- OpenAI-compatible API (optional for LLM mode)
- Google Gemini API (optional)
- Transformers (optional for local sentiment analysis)
- Rule-based fallback system (ensures system works without API key)

Frontend Dependencies

- React
- Vite
- Tailwind CSS

5. Environment Configuration

The system requires configuration of environment variables for proper functioning:

- OPENAI_API_KEY – API key for LLM-based response generation
- OPENAI_MODEL – model configuration
- OPENAI_BASE_URL – API endpoint (if applicable)
- DATABASE_URL – SQLite database path
- CORS_ORIGINS – allowed frontend origins

This ensures secure and flexible communication between system components.

6. System Setup

To run the project locally:

Backend Setup:

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Frontend Setup:

```
npm install  
npm run dev
```

Ensure both frontend and backend servers are running for proper communication.

7. Deployment Requirements

- **Deployment Platform:** Vercel (used in this project)
- Proper build configuration for frontend
- Backend API deployment or serverless integration
- Internet access for live usage

8. Optional Requirements

- API key for advanced AI response generation
- Cloud database (e.g., PostgreSQL) for scaling
- GPU support (only if running local transformer models)

Libraries / Frameworks / Dependencies

- The development of *ToneCraft AI: Emotion-Aware Response Composer* relies on a combination of backend frameworks, frontend libraries, AI/NLP tools, and supporting dependencies. These components enable efficient system performance, scalability, and intelligent response generation.
-

• 1. Backend Libraries and Frameworks

- **FastAPI**
A modern, high-performance web framework used to build RESTful APIs. It handles request routing, asynchronous processing, and integration with AI services.
- **Uvicorn**
An ASGI server used to run the FastAPI application efficiently with support for asynchronous operations.
- **Pydantic**
Used for data validation and parsing. It ensures that all incoming and outgoing data follows a defined schema.
- **SQLAlchemy**
An Object Relational Mapper (ORM) used to interact with the SQLite database. It simplifies database operations such as storing and retrieving cases.
-

• 2. Artificial Intelligence and NLP Dependencies

- **OpenAI-Compatible API (Optional)**
Used for generating high-quality, context-aware responses using Large Language Models (LLMs).
- **Google Gemini API (Optional)**
Alternative LLM provider used for generating intelligent responses.
- **Transformers (Optional)**
Used for local sentiment analysis and emotion detection when external APIs are not available.
- **Rule-Based Fallback System**
A custom logic-based approach used when AI APIs are unavailable. It ensures the system can still generate safe and empathetic responses.
-

• 3. Frontend Libraries and Frameworks

- **React**
A JavaScript library used for building dynamic and interactive user interfaces.
- **Vite**
A fast frontend build tool used to develop and bundle the React application efficiently.
- **Tailwind CSS**
A utility-first CSS framework used for designing responsive and modern UI components.
-

• 4. Testing Framework

- **pytest**
A testing framework used to validate backend functionality. It ensures that APIs and system workflows operate correctly.
-

• 5. Supporting Dependencies

- **JSON**

Used as the standard format for data exchange between frontend and backend.

- **CORS Middleware**

Enables secure communication between frontend and backend running on different origins.

-

Hardware Requirements

The *ToneCraft AI: Emotion-Aware Response Composer* is a lightweight, web-based application and does not require high-end or specialized hardware. Since the system primarily relies on a FastAPI backend, React frontend, and optional external AI APIs, it can run efficiently on standard computing devices.

1. Minimum Hardware Requirements

The system can be developed and executed on a basic computer with the following configuration:

- **Processor:** Dual-core CPU (e.g., Intel i3 or equivalent)
- **RAM:** 4 GB
- **Storage:** At least 1 GB of free disk space
- **Internet Connection:** Required for API access, package installation, and deployment

2. Recommended Hardware Requirements

For smoother development, faster processing, and better multitasking:

- **Processor:** Intel i5 / Ryzen 5 or higher
- **RAM:** 8 GB or more
- **Storage:** SSD with at least 5 GB free space
- **Internet Connection:** Stable broadband connection

3. Hardware for Deployment

Since the project is deployed using **Vercel**, no dedicated physical server hardware is required by the developer. The hosting platform manages:

- Server infrastructure
- Scaling and availability
- Request handling

4. AI Processing Considerations

- The project primarily uses **API-based AI models (LLMs like OpenAI/Gemini)**

- No GPU is required
- In case of optional local model usage (Transformers):
 - CPU-based execution is sufficient for basic sentiment analysis
 - GPU is **not mandatory** and not used in the current implementation

5. User-Side Requirements

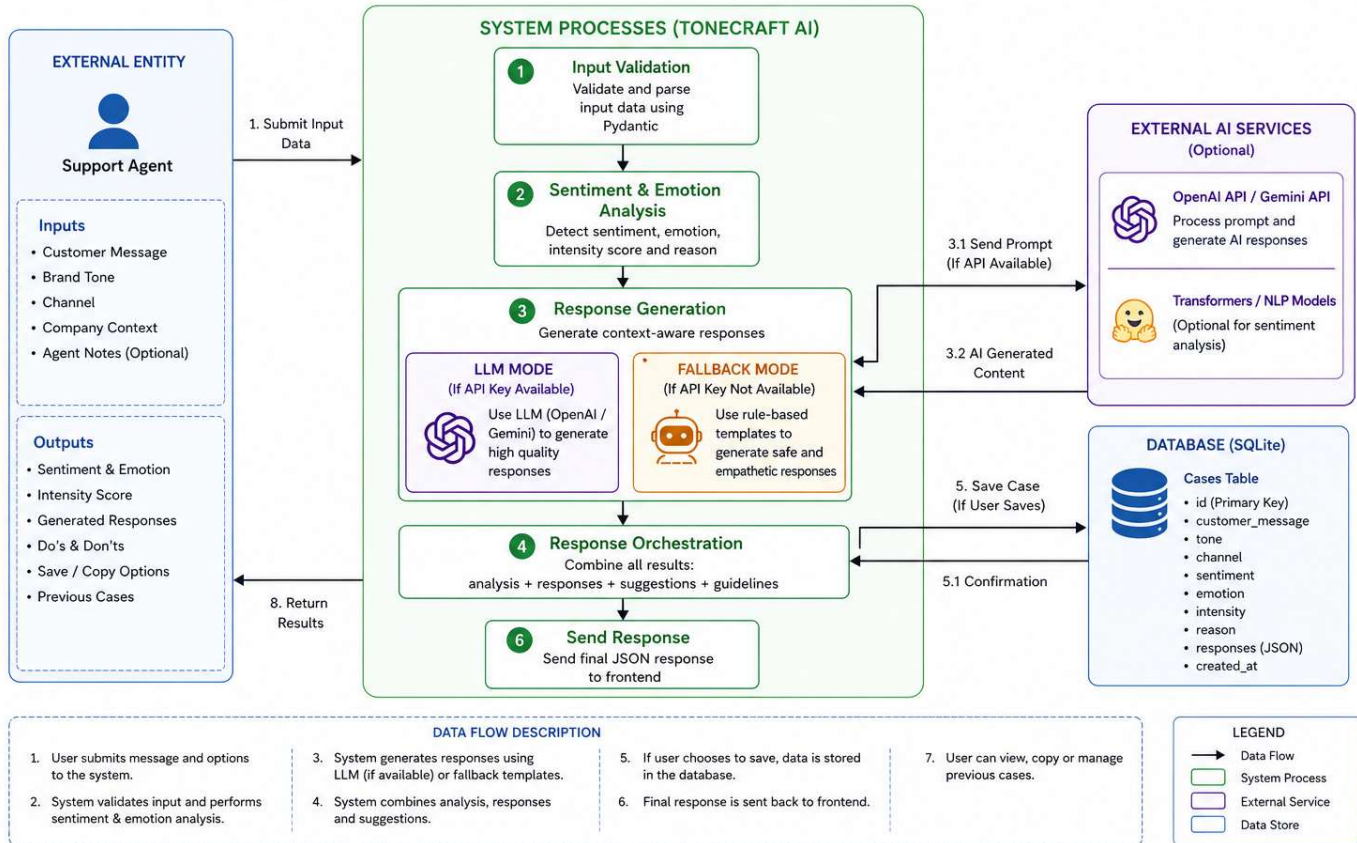
End users (support agents) only need:

- A device (laptop, desktop, or mobile)
- A web browser (Chrome, Edge, etc.)
- Internet connection

No installation or special hardware is required on the user side.

TONECRAFT AI – EMOTION-AWARE RESPONSE COMPOSER

DATA FLOW DIAGRAM (LEVEL 1)



• PRIOR KNOWLEDGE REQUIRED

To effectively understand, develop, and work with *ToneCraft AI: Emotion-Aware Response Composer*, a basic level of knowledge in the following areas is required. These concepts are directly related to the technologies and implementation used in the project.

1. Programming Fundamentals

A basic understanding of programming is essential, particularly:

- **Python** – used for backend development, API creation, and AI integration
- **JavaScript** – used for frontend development and user interaction

Knowledge of variables, functions, loops, and basic logic building is sufficient.

2. Web Development Concepts

Understanding how web applications work is important for this project:

- **HTML & CSS** – for structuring and styling the user interface
- **React** – for building dynamic frontend components
- **Vite** – for fast frontend development and bundling

This helps in understanding how the user interface interacts with the backend.

3. Backend and API Development

Basic knowledge of backend systems and APIs is required:

- **FastAPI framework** – used to build RESTful APIs
- **REST API concepts** – request-response cycle (GET, POST, etc.)
- **JSON format** – for data exchange between frontend and backend
- **Pydantic** – for data validation

4. Database Concepts

Understanding how data is stored and retrieved:

- **SQLite** – lightweight database used in the project
- **SQLAlchemy** – ORM for managing database operations

- Basic concepts of tables, records, and queries

5. Artificial Intelligence and NLP Basics

Basic knowledge of AI concepts used in the system:

- **Natural Language Processing (NLP)** – processing and understanding text
- **Sentiment Analysis** – detecting positive, negative, or neutral tone
- **Emotion Detection** – identifying emotions like anger, frustration, etc.
- **Generative AI / LLMs** – generating human-like responses

6. Prompt-Based AI Interaction

Understanding how prompts are used to interact with AI models:

- Structuring input text for better responses
- Understanding how AI generates output based on context

7. Software Development Concepts

Basic understanding of:

- Client-server architecture
- Frontend and backend interaction
- Modular coding structure

8. Basic Debugging and Testing

- Ability to run applications locally
- Understanding logs and error messages
- Basic testing concepts (e.g., using pytest)

Project Workflow

The workflow of *ToneCraft AI: Emotion-Aware Response Composer* is designed as a **modular, service-oriented pipeline** that processes customer input through validation, analysis, response generation, and presentation layers. The system ensures reliability through a **dual-processing architecture (LLM + fallback)** and maintains consistency via structured orchestration.

1. User Interaction and Context Formation (Frontend Layer)

The workflow begins with the support agent interacting with the **React-based UI**. The system collects structured inputs:

- Customer message (primary input)
- Brand tone (Professional, Warm, Apologetic, etc.)
- Communication channel (Email, Chat, WhatsApp, Social Media)
- Optional company context (policies, service info)
- Optional agent notes

👉 Why this step is important:

This transforms raw text into **context-rich input**, which significantly improves response accuracy.

2. Request Packaging and API Communication

The frontend converts the input into a structured JSON payload:

```
{  
  "message": "...",  
  "tone": "...",  
  "channel": "...",  
  "company_context": "...",  
  "agent_notes": "..."  
}
```

This is sent to:

- **Endpoint:** /api/analyze
- **Protocol:** HTTP (REST API)

👉 The system uses asynchronous communication for fast response handling.

3. Backend Entry Point and Validation Layer

The request reaches the FastAPI backend:

- Handled by: `analyze.py` router
- Validated using: **Pydantic schemas**

Validation checks include:

- Required fields present
- Data types correct
- Input length sanity

👉 Failure Handling:

Invalid inputs are rejected early → improves system reliability.

4. Service Orchestration Layer (Core Control Unit)

The request is passed to:

👉 `response_service.py`

This acts as the **central orchestrator**, coordinating:

- Sentiment analysis
- Emotion detection
- Response generation
- Output formatting

👉 Think of this as the **brain of the system flow**

5. Sentiment and Emotion Analysis Pipeline

Handled by: `sentiment_service.py`

Multi-layer Processing:

A. Transformer-Based Model (Optional)

- If available, uses NLP pipeline
- Provides better contextual understanding

B. Rule-Based Fallback System

Used when transformer is unavailable:

- Keyword detection logic

- Pattern-based scoring

Example Mapping:

- “frustrating”, “again” → Frustration
- “angry”, “unacceptable” → Anger
- “urgent”, “ASAP” → Urgency

Generated Output:

- Sentiment → Positive / Neutral / Negative
- Emotion → Specific emotional state
- Intensity → Score (0–100)
- Reason → Explanation for detection

👉 Why important:

This step drives the **tone and structure of generated responses**

6. Intelligent Response Generation Engine

Handled by: `llm_service.py`

The system dynamically selects one of two modes:

A. LLM Mode (Primary Path)

Condition:

✓ OPENAI_API_KEY available

Flow:

1. Load prompt from `response_prompt.py`
2. Inject:
 - a. Message
 - b. Emotion
 - c. Tone
 - d. Channel
3. Send request to:
 - a. OpenAI-compatible API OR Gemini

Output:

- Strict JSON response
- High-quality, context-aware replies

B. Fallback Mode (Critical Reliability Layer)

Condition:

✗ API key missing OR API failure

Flow:

- Uses rule-based templates
- Applies:
 - Emotion-aware phrasing
 - Channel adaptation
 - Brand tone alignment

Guarantees:

- System never crashes
- Always produces output

7. Response Structuring and Enhancement

After generation, the system constructs a unified response:

Includes:

- Sentiment
- Emotion
- Intensity
- Reason
- Tone adjustment suggestion

Response Variants:

- Main reply
- Short reply
- Detailed reply
- De-escalation reply

Guidance:

- Do's (best practices)
- Don'ts (avoid mistakes)

👉 This creates a **complete decision-support package for agents**

8. Database Interaction Layer (Case Management)

Handled via:

- `models.py`
- `database.py`
- `cases.py` router

Trigger:

Only when user clicks **Save Case**

Stored Data:

- Input parameters
- Analysis results
- Generated responses
- Timestamp

Operations:

- Save → POST `/api/cases`
- Retrieve → GET `/api/cases`
- Fetch → GET `/api/cases/{id}`
- Delete → DELETE `/api/cases/{id}`

👉 SQLite is used for:

- Lightweight storage
- Local/demo usage

9. Backend Response Construction

Final output is packaged into JSON:

```
{  
  "sentiment": "...",  
  "emotion": "...",  
  "intensity": ...,  
  "reason": "...",  
  "responses": {...},  
  "guidance": {...}  
}
```

Returned to frontend via HTTP response.

10. Frontend Rendering and Interaction

Handled by:

- ResultPanel.jsx
- SavedCases.jsx

Features:

- Emotional intensity progress bar
- Multiple response display
- Copy-to-clipboard buttons
- Save case functionality
- Case history view

11. Continuous Feedback Loop

The system supports iterative usage:

- Agent refines inputs
- Compares responses
- Reuses saved cases

👉 This improves:

- Agent efficiency
- Response consistency
- Learning curve

End-to-End Workflow Summary

1. Agent inputs message + context
2. Frontend sends structured request
3. Backend validates input
4. Orchestrator triggers services
5. Sentiment & emotion analysis performed
6. Response generated (LLM or fallback)
7. Results structured with guidance
8. Data optionally stored in database
9. Backend returns JSON response
10. Frontend displays and enables actions

System Architecture Overview

The system architecture of *ToneCraft AI: Emotion-Aware Response Composer* is designed using a modular, layered, and service-oriented approach to ensure scalability, reliability, and efficient processing. The architecture consists of a React-based frontend for user interaction, a FastAPI backend for handling API requests and business logic, an AI/NLP processing layer for sentiment analysis and intelligent response generation, and a SQLite database for storing and managing cases. The backend is further divided into structured modules such as routes, services, models, utilities, and database layers, enabling clear separation of concerns and maintainability. The system supports dual processing modes—LLM-based generation using OpenAI-compatible or Gemini APIs, and a rule-based fallback mechanism—ensuring continuous operation even without external AI access. Communication between components is handled through RESTful APIs, allowing real-time data exchange and response generation. Overall, the architecture is designed to be robust, flexible, and deployment-friendly, making it suitable for both academic use and real-world customer support applications.

1. Backend Overview

The backend of *ToneCraft AI* is built using **FastAPI** and follows a modular, service-oriented architecture. It is responsible for handling API requests, performing sentiment and emotion analysis, generating responses, and managing data storage.

The backend ensures:

- Efficient request handling
- Clear separation of logic
- Integration with AI models (LLM or fallback)
- Reliable data processing and storage

2. Backend Modular Architecture (backend/app/)

The backend is organized into multiple modules to ensure scalability and maintainability. Each module has a specific responsibility, enabling clean code structure and easier debugging.

3. Routes Layer (backend/app/routers/)

This layer defines all API endpoints and acts as the entry point for client requests.

Key Responsibilities:

- Handling HTTP requests
- Routing requests to appropriate services
- Returning structured responses

Main Routes:

- `/api/analyze` → Performs sentiment analysis and response generation
- `/api/cases` → Handles saving, retrieving, and deleting cases
- `/health` → Checks system status

4. Services Layer (backend/app/services/)

The services layer contains the core business logic of the system.

Key Components:

- **sentiment_service.py**
Performs sentiment and emotion detection using transformer models or rule-based fallback.
- **llm_service.py**
Handles communication with external AI models (OpenAI/Gemini) and manages prompt-based response generation.
- **response_service.py**
Acts as the central orchestrator, combining analysis and response generation into a final output.

Responsibilities:

- Processing input data
- Applying AI/NLP logic
- Generating structured results

5. Models Layer (backend/app/models/)

This layer defines the database schema using SQLAlchemy.

Responsibilities:

- Defining structure of stored data
- Representing customer cases
- Managing database entities

Stored Fields Include:

- Customer message
- Sentiment and emotion
- Intensity score
- Generated responses
- Timestamp

6. Utils Layer (backend/app/utils/)

The utils layer contains helper functions and reusable logic.

Responsibilities:

- Data formatting
- Common utility functions
- Error handling support

- Helper methods for processing

This layer improves code reusability and reduces redundancy.

7. Database Layer (backend/app/db/)

This layer manages database connectivity and operations.

Technologies Used:

- SQLite (lightweight database)
- SQLAlchemy (ORM)

Responsibilities:

- Establishing database connection
- Managing sessions
- Initializing database tables
- Performing CRUD operations

8. Frontend Overview

The frontend is built using **React, Vite, and Tailwind CSS** and provides an interactive interface for users.

Responsibilities:

- Collecting user input
- Sending API requests
- Displaying analysis and generated responses
- Managing user interactions

9. Frontend Structure (frontend/src/)

The frontend follows a component-based architecture for better modularity.

10. API Layer (frontend/src/api/)

This layer handles communication between frontend and backend.

Responsibilities:

- Sending HTTP requests
- Receiving API responses
- Managing endpoints

Example:

- Call to `/api/analyze`
- Call to `/api/cases`

11. Components (frontend/src/components/)

Reusable UI components used across the application.

Key Components:

- **AnalyzeForm.jsx** → Input form for user data
- **ResultPanel.jsx** → Displays output results
- **SavedCases.jsx** → Displays stored cases
- **Header.jsx** → UI header

12. Pages (frontend/src/pages/)

Defines main views or layouts of the application.

Responsibilities:

- Organizing components into full pages
- Managing navigation and layout

13. Utils (frontend/src/utils/)

Contains helper functions for frontend logic.

Responsibilities:

- Data formatting
- Utility functions
- Reusable logic for UI behavior

14. Supabase & Database Migrations

In the current implementation:

- **SQLite** is used for backend storage
- Supabase is **not actively used** in the deployed system

Note:

Future improvements may include:

- Migration to cloud databases like Supabase or PostgreSQL
- Better scalability and multi-user support

15. Deployment Configuration

The system is deployed using **Vercel**, ensuring fast and scalable hosting.

Deployment Details:

- Frontend hosted on Vercel
- Backend API deployed and connected
- Environment variables configured for API access

Special Note:

For reliability in serverless deployment:

- Case storage may be handled on frontend side (browser persistence)

16. Documentation (README.md)

The project includes a README file that provides:

- Project overview
- Setup instructions
- API usage details
- Folder structure explanation
- Deployment guidance

This helps developers understand and run the project easily.

RESULTS

The *ToneCraft AI: Emotion-Aware Response Composer* was successfully designed, developed, and deployed as a functional web-based application. The system demonstrates the ability to analyze customer messages, detect emotional context, and generate appropriate responses in real time.

1. Functional Results

The system was tested with various types of customer messages, including complaints, queries, and urgent requests. The following outcomes were observed:

- The system accurately identifies **sentiment** as Positive, Neutral, or Negative
- It successfully detects **emotional states** such as Anger, Frustration, Confusion, and Urgency
- It assigns a meaningful **intensity score (0–100)** along with a clear explanation
- It generates **multiple response variations**, including:
 - Main reply
 - Short reply
 - Detailed reply
 - De-escalation reply

The generated responses were found to be:

- Context-aware
- Emotionally appropriate
- Professionally structured

2. Dual-Mode Reliability (LLM + Fallback)

One of the key results of the project is its **reliability under different conditions**:

- When an API key is available:
 - The system uses LLMs (OpenAI/Gemini) to generate high-quality responses
- When API access is unavailable:
 - The system switches to a rule-based fallback mechanism



The application **never fails to generate output**, ensuring consistent usability.

Result:

3. Performance Results

- The system provides **near real-time response generation**
- FastAPI backend ensures **low latency API communication**
- Frontend UI updates instantly with results

The overall performance is efficient for real-world usage without requiring high computational resources.

4. User Interface and Experience

The deployed application provides a clean and interactive interface:

- Easy input form for message and context
- Clear visualization of:
 - Sentiment
 - Emotion
 - Intensity (progress bar)
- One-click copy functionality
- Case saving and retrieval features



The system is **user-friendly and suitable for support agents with minimal training**

Result:

5. Testing Results

Backend testing was conducted using **pytest**, covering:

- Health endpoint functionality
- Analysis endpoint correctness
- Case management operations (create, fetch, delete)

Test Outcome:

- All core test cases passed successfully
- System behaved as expected under different input scenarios

6. Deployment Results

The application was successfully deployed using **Vercel**:

- Frontend is live and accessible
- Backend API is functional
- System works in a serverless environment

Special Observation:

- For deployment reliability, case storage is partially handled on the frontend side

7. Practical Effectiveness

The system effectively achieves its main goals:

- Reduces response drafting time
- Improves empathy in communication
- Maintains tone consistency
- Assists in de-escalating difficult conversations

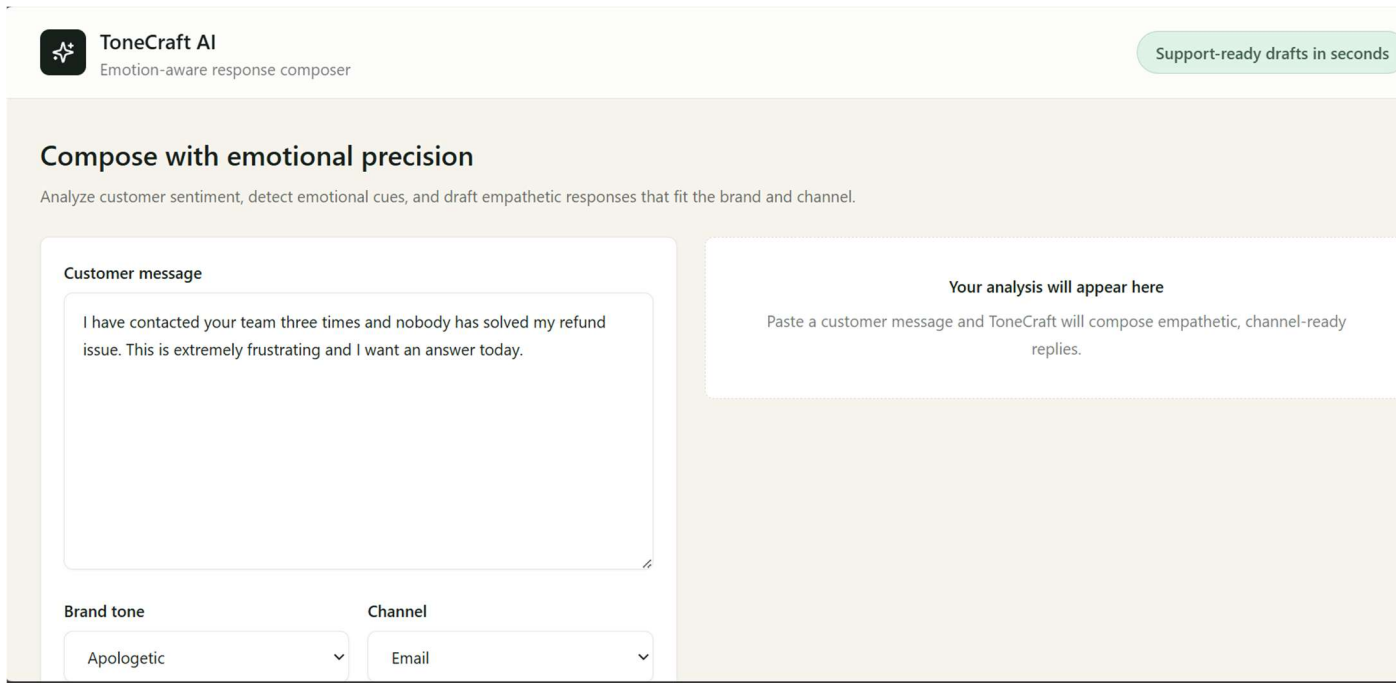
8. Sample Output Behavior (Observed)

For a frustrated customer message:

- Emotion detected: **Frustration**
- Sentiment: **Negative**
- System response:
 - Includes apology
 - Acknowledges issue
 - Provides next steps
 - Avoids blame and overpromising

👉 This demonstrates real-world applicability.

● Screenshots and User Interface Representation



ToneCraft AI
Emotion-aware response composer

Support-ready drafts in seconds

Compose with emotional precision

Analyze customer sentiment, detect emotional cues, and draft empathetic responses that fit the brand and channel.

Customer message

I have contacted your team three times and nobody has solved my refund issue. This is extremely frustrating and I want an answer today.

Your analysis will appear here

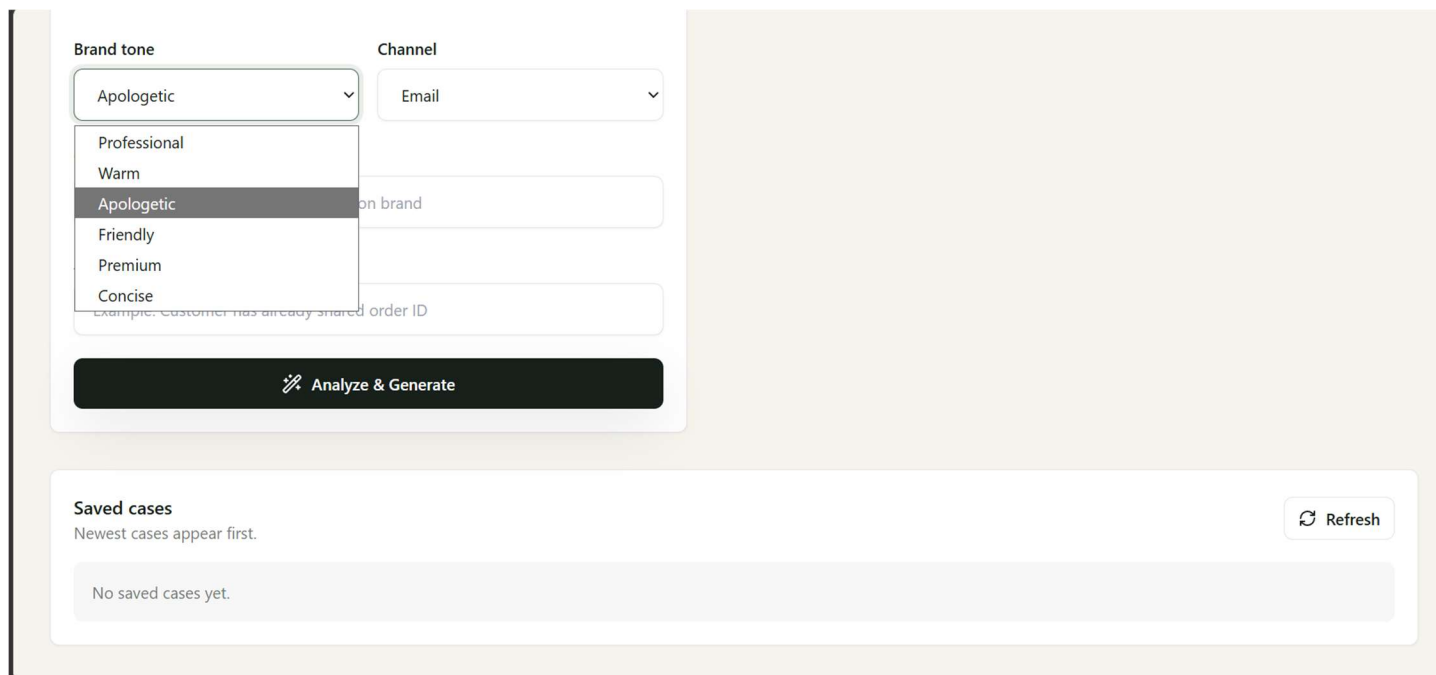
Paste a customer message and ToneCraft will compose empathetic, channel-ready replies.

Brand tone

Apologetic

Channel

Email



Brand tone

Apologetic

Professional

Warm

Apologetic

Friendly

Premium

Concise

Channel

Email

on brand

Example: Customer has already shared order ID

Analyze & Generate

Saved cases

Newest cases appear first.

Refresh

No saved cases yet.

The user interface of *ToneCraft AI: Emotion-Aware Response Composer* is designed as a clean, structured dashboard that enables support agents to input customer messages, configure response settings, and view AI-generated outputs efficiently.

1. Main Dashboard Layout

The application interface is divided into two primary sections:

- **Left Section (Input Panel)**
- **Right Section (Output Panel)**

At the top, the application displays:

- Project title: **“ToneCraft AI”**
- Subtitle: *Emotion-aware response composer*
- Tagline: *Support-ready drafts in seconds*

Below this, a heading is shown:

- **“Compose with emotional precision”**

This is followed by a short description explaining that the system analyzes sentiment and generates empathetic responses.

2. Customer Message Input Section

The left panel contains a large text input area labeled:

“Customer message”

This field allows the user to paste or type customer queries or complaints.

Example shown in the interface:

“I have contacted your team three times and nobody has solved my refund issue. This is extremely frustrating and I want an answer today.”

This message demonstrates a real-world frustrated customer scenario.

3. Configuration Options

Below the message input, the system provides configuration controls:

a. Brand Tone Selection

A dropdown labeled **“Brand tone”** is available with the following options:

- Professional
- Warm
- Apologetic
- Friendly
- Premium
- Concise

In the screenshot, **“Apologetic”** is selected, indicating that the response will include empathy and apology.

b. Channel Selection

A dropdown labeled **“Channel”** allows the user to select the communication medium:

- Email (selected in the screenshot)
- Other possible channels include chat, WhatsApp, or social media

This ensures the generated response matches the communication format.

c. Additional Context Fields

Optional input fields are provided for:

- Company context
- Agent notes

These fields help refine response generation by adding business-specific or situational details.

4. Action Button

A prominent button labeled:

“Analyze & Generate”

This button triggers the complete workflow:

- Sentiment analysis
- Emotion detection
- Response generation

5. Output Panel (Right Section)

The right side of the interface displays the result area.

Before processing, it shows a placeholder message:

“Your analysis will appear here”

“Paste a customer message and ToneCraft will compose empathetic, channel-ready replies.”

After clicking the button, this section displays:

- Sentiment result
- Emotion detected
- Intensity score
- Generated responses

6. Saved Cases Section

Below the input area, the interface includes a **Saved Cases panel**.

Features:

- Displays previously analyzed cases
- Shows message: **“No saved cases yet”** when empty
- Includes a **“Refresh” button**
- Displays cases in **newest-first order**

This allows users to revisit past interactions.

7. User Experience Design

The UI follows modern design principles:

- Clean and minimal layout
- Clear separation between input and output
- Easy-to-use dropdowns and text fields
- Logical flow from input → action → output
- Responsive design for smooth interaction

Summary

The user interface of ToneCraft AI is designed to provide a seamless experience for support agents by combining structured input, configurable response settings, and real-time output display. The layout ensures clarity, ease of use, and efficient workflow execution, making it suitable for practical customer support scenarios.

ADVANTAGES & LIMITATIONS

The *ToneCraft AI: Emotion-Aware Response Composer* offers several practical benefits for improving customer communication. However, like any system, it also has certain limitations based on its current implementation.

Advantages

1. Emotion-Aware Communication

The system analyzes customer sentiment and emotional state, enabling the generation of responses that are empathetic and contextually appropriate. This improves the quality of customer interactions.

2. Dual Processing Mode (LLM + Fallback)

The application supports:

- AI-based response generation using LLM APIs
- Rule-based fallback mechanism

👉 This ensures the system works even without external API access, making it highly reliable.

3. Real-Time Response Generation

The system generates responses instantly after user input, reducing the time required for drafting replies and improving productivity.

4. Multiple Response Variations

The system provides:

- Main reply
- Short reply
- Detailed reply

- De-escalation reply

This gives flexibility to agents to choose the most suitable response.

5. Brand Tone Customization

Users can select different tones such as:

- Professional
- Warm
- Apologetic
- Friendly
- Premium
- Concise

This ensures consistency in communication across different scenarios.

6. Multi-Channel Support

The system adapts responses based on communication channels such as:

- Email
- Chat
- WhatsApp
- Social media

7. Modular and Scalable Architecture

The backend is structured into services and modules, making it:

- Easy to maintain
- Extendable for future features
- Suitable for integration with other systems

8. Lightweight and Cost-Effective

- Does not require GPU
- Runs on standard hardware
- Uses SQLite for simple storage

9. User-Friendly Interface

The clean and structured UI allows:

- Easy input of messages
- Simple configuration
- Quick response access

10. Deployment Ready

The system is deployed on Vercel and works in a serverless environment, making it easy to access and demonstrate.

Limitations

1. Rule-Based Emotion Detection (Fallback Mode)

When LLM or transformer models are not used, emotion detection relies on keyword-based logic, which may not fully capture complex or subtle emotions.

2. Limited Database Scalability

The system uses SQLite, which is suitable for:

- Single-user or demo environments

However, it is not ideal for:

- Large-scale multi-user systems

3. Dependency on External APIs (LLM Mode)

Advanced response quality depends on:

- Availability of API key
- Internet connectivity

If API fails, the system falls back to basic templates.

4. Limited Context Awareness

The system primarily processes:

- Single-message input

It does not maintain:

- Full conversation history
- Multi-turn dialogue context

5. No Authentication System

Currently, the system does not include:

- User login
- Role-based access control

This limits its use in enterprise environments.

6. Frontend Testing Coverage

Backend testing is implemented, but:

- Frontend unit/integration tests are not included

7. Deployment Storage Constraints

In deployed mode:

- Case storage may rely on frontend/browser persistence
- Backend SQLite may not fully persist across sessions in serverless environments

8. Limited Personalization

The system uses predefined tone options but does not support:

- Fully customizable brand voice profiles
- Adaptive learning from user feedback

Backup and Recovery

The *ToneCraft AI: Emotion-Aware Response Composer* system incorporates a basic but effective backup and recovery approach suitable for its current architecture and deployment environment. Since the system uses a lightweight database and is deployed in a serverless-friendly setup, backup strategies are designed to ensure data safety and system reliability without complex infrastructure.

1. Data Storage Overview

The system stores application data primarily in:

- **SQLite Database (Backend)**
Used to store customer cases, including:
 - Customer message
 - Tone and channel
 - Sentiment and emotion
 - Generated responses
 - Do's and Don'ts guidance
 - Timestamp
- **Frontend (Browser Storage – Deployment Mode)**
In the deployed environment, case data may be stored on the frontend side to ensure persistence when backend storage is limited.

2. Backup Mechanism

a. SQLite Database Backup

- SQLite stores all data in a single file
- Backup can be performed by:
 - Copying the database file manually
 - Creating periodic snapshots of the file

👉 This makes backup simple and efficient for small-scale systems.

b. Frontend Data Backup

- In deployment mode, saved cases may persist in browser storage
- Data remains available as long as browser storage is not cleared

👉 This acts as a temporary backup mechanism in serverless environments.

3. Recovery Mechanism

a. Database Recovery

- Recovery is performed by restoring the SQLite database file from backup
- Once restored, all stored cases and records become accessible again

b. Application Recovery

- The system is stateless in nature (except for saved cases)
- In case of failure:
 - Restarting backend services restores functionality
 - Frontend reload reconnects to backend APIs

c. Fallback System Recovery

- If AI services fail (e.g., API unavailable):
 - The system automatically switches to **fallback mode**
 - Ensures continuous operation without interruption

4. Reliability Features

- **Dual-mode processing (LLM + fallback)** ensures system availability
- **Stateless API design** reduces risk of data corruption
- **Lightweight database structure** simplifies recovery

5. Limitations in Backup and Recovery

- SQLite does not support advanced backup features like:
 - Automatic replication
 - Distributed storage
- Browser-based storage is:
 - Temporary
 - Not reliable for long-term persistence

6. Future Improvements

To enhance backup and recovery capabilities, the following improvements can be implemented:

- Use **cloud databases** (e.g., PostgreSQL, Supabase)
- Implement **automated backup scheduling**
- Add **data synchronization across devices**
- Introduce **secure cloud storage for persistent data**
- Enable **user-based data management and recovery**

CONCLUSION

The ***ToneCraft AI: Emotion-Aware Response Composer*** successfully demonstrates how artificial intelligence can enhance customer communication by combining sentiment analysis, emotion detection, and intelligent response generation. The system effectively transforms raw customer messages into empathetic, context-aware, and professionally structured replies, helping support agents handle interactions more efficiently.

The project achieves its primary objectives by accurately identifying customer sentiment and emotional state, generating multiple response variations, and providing tone adjustment guidance. The integration of a dual-mode processing system—LLM-based generation and rule-based fallback—ensures that the application remains reliable and functional even in the absence of external AI services.

The modular architecture, built using FastAPI for the backend and React for the frontend, enables clear separation of concerns, maintainability, and scalability. The system's lightweight design allows it to run on standard hardware, while its deployment on Vercel demonstrates its practicality in real-world environments.

Additionally, the user-friendly interface simplifies interaction, making the system accessible even to users with minimal technical expertise. Features such as tone customization, multi-channel support, and case management further enhance its usability and relevance in customer support scenarios.

However, the project also highlights areas for improvement, including enhanced emotion detection accuracy, better scalability of the database, and support for multi-turn conversational context. These limitations provide opportunities for future development and refinement.

In conclusion, ToneCraft AI presents a practical and effective solution for improving communication quality in customer support systems. It bridges the gap between human emotional understanding and automated systems, making it a valuable tool for both academic research and real-world application.

Each API endpoint includes:

- Defined request parameters
- Structured JSON responses
- Error handling and validation mechanisms

The API design ensures seamless communication between frontend and backend, and also allows integration with external systems if needed.

GitHub Repository Link

The complete project source code, along with documentation and setup instructions, is available in the GitHub repository.

Repository Link:

<https://github.com/rishik1072/ToneCraft-AI>

The repository includes:

- Full source code (frontend and backend)
- Installation and setup instructions
- Configuration guidelines
- Project documentation

This allows other developers and evaluators to explore, run, and contribute to the project.